

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

Periodic Real-Time Scheduling Simulator: RM, DM, and EDF

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Embedded Systems

Contents

1 Embedded architectures 2

2 Real-time versus general-purpose operating systems 2

3 Priority inversion and resource protocols 2

4 Hard and soft deadlines 2

5 Fault-tolerance techniques 3

6 TinyML and edge AI 3

7 Weighted CPI, execution time, and MIPS 3

8 Simulator input and priority rules 3

9 Recorded one-core and two-core outcomes 6

10 Schedulability analysis 9

1 Embedded architectures

A single-processor system simplifies implementation, communication, and scheduling, and can suit household controls, simple IoT sensors, and small controllers. Its limited compute capacity makes substantial parallel work difficult. Execution variability and power consumption depend on interrupts, caches, peripherals, clock settings, and workload. Multicore architectures allow concurrent computation and are useful for image processing and advanced driver-assistance workloads. Shared-memory synchronization, inter-core communication, cache interference, and scheduling complicate analysis. More active cores can increase power, but an energy comparison requires a measured workload and power policy. A system-on-chip integrates processors, interfaces, peripherals, and possibly accelerators and memory on one device. Integration reduces board size and enables close coupling. Custom chip design can have substantial nonrecurring engineering cost and limited hardware upgradeability; purchasing an existing SoC is a different cost model. Smartphones, networking equipment, and drones illustrate integrated platforms. "SoC" describes integration, so it can coexist with either a single-core or multicore processor.

$$U = \sum_{i} C_i/T_i, \qquad R_i = C_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j.$$

2 Real-time versus general-purpose operating systems

An RTOS is designed to support bounded, analyzable response times under specified assumptions. Priority-based preemption is common, but an RTOS does not automatically make every application meet its deadlines. General-purpose systems balance throughput, fairness, responsiveness, and many concurrent services, which can make worst-case response harder to bound. Fixed-size memory pools and avoidance of demand paging help limit timing variability. General-purpose virtual-memory operations can introduce page faults and reclamation delays. Real-time designs also minimize and bound interrupt latency; hardware, interrupt priorities, drivers, critical sections, and configuration affect that bound. These distinctions motivate explicit timing analysis rather than inferring guarantees from an operating-system label.

3 Priority inversion and resource protocols

Priority inversion occurs when a high-priority job waits for a lock held by a low-priority job, while a medium-priority job preempts the lock holder and prolongs the high-priority wait. Priority inheritance temporarily boosts the lock holder to the waiting job's priority so it can release the resource sooner. Chained dependencies and lock discipline still matter. Ceiling protocols assign each resource a ceiling based on the highest-priority task that can use it. Immediate priority elevation on locking corresponds to an immediate-ceiling formulation. Under their specified assumptions, ceiling protocols can bound blocking and prevent certain deadlock patterns. Priority inheritance and the different ceiling formulations must not be treated as interchangeable or unconditional deadlock cures.

4 Hard and soft deadlines

A hard real-time requirement treats exceeding a deadline as unacceptable for the function being assured. Automotive braking and airbag-control functions illustrate safety-critical timing needs, although the exact permissible interval depends on the specification and is not established by the report's illustrative 100-ms number. Soft real-time services tolerate occasional lateness with degraded quality, as in video streaming or online games. A missed deadline does not necessarily imply

a physical catastrophe in every hard real-time system; it violates the required timing contract.

5 Fault-tolerance techniques

Redundancy provides alternate components or replicated computation when a primary path fails, at the cost of hardware, power, voting/coordination, and possible common-cause failures. A watchdog detects failure to make expected progress and triggers recovery; it does not repair the underlying software bug and must be serviced only when meaningful progress is established. Checkpointing stores state for restart after a fault, with storage and timing overhead and a need to validate the checkpoint. Parity and CRC detect specified classes of corruption; they add bits and processing and do not detect every possible error or provide authentication. These four mechanisms address different failure modes.

6 TinyML and edge AI

Small embedded platforms may have only tens of kilobytes of RAM, limited arithmetic capability, and tight battery budgets. Pruning removes selected network parameters or structures, although unstructured sparsity needs supporting execution software to realize a speedup. Quantization can replace 32-bit floating-point weights with eight-bit representations, reducing weight storage by roughly four times before metadata and mixed-precision overhead. Its accuracy loss must be evaluated rather than assumed negligible. Distillation trains a smaller student from a larger teacher. These techniques trade precision, capacity, implementation complexity, and energy; none makes every model fit an arbitrary device.

7 Weighted CPI, execution time, and MIPS

For category weights 0.4, 0.3, 0.1, and 0.2, case 1 gives $2(0.4)+3(0.3)+4(0.1)+5(0.2)=3.1$ cycles per instruction. Case 2 gives $3(0.4)+2(0.3)+3(0.1)+4(0.2)=2.9$. Execution time is instruction count multiplied by CPI and divided by clock frequency. MIPS is frequency divided by CPI and by one million.

Processor	**Program instructions (million)**	**Clock (GHz)**	**Time case 1 (ms)**	**Time case 2 (ms)**	**MIPS case 1**	**MIPS case 2**
A	12	2.0	18.6	17.4	645.16	689.66
B	9	1.5	18.6	17.4	483.87	517.24

The programs finish in equal times in this calculation because their instruction-count/frequency ratios are equal. Their MIPS differs. MIPS is not an architecture-independent measure of useful work when instruction sets or program instruction counts differ.

8 Simulator input and priority rules

The supplied 30-unit experiment uses T1=(C=1,T=4,D=4), T2=(3,8,6), and T3=(1,7,7). It is run with one core and then two. RM gives shorter periods higher fixed priority, so T1 precedes T3 and T2. DM orders by relative deadline, so T1 precedes T2 and T3. EDF chooses the ready job with the earliest absolute deadline, which can change with releases.

```python
import matplotlib.pyplot as plt

def simulate_scheduling(algorithm, num_cores, total_time, tasks):
    ready_queue = []
    completed_jobs = 0
    deadline_misses = 0
    busy_cycles = 0
    schedule = {i: [] for i in range(num_cores)}

    for current_time in range(total_time):
        for task in tasks:
            if current_time % task[2] == 0:
                ready_queue.append([task[0], task[1], current_time + task[3], task[2], task[3]])

        next_queue = []
        for job in ready_queue:
            if current_time == job[2] and job[1] > 0:
                deadline_misses += 1
            elif job[1] > 0:
                next_queue.append(job)
        ready_queue = next_queue

        if algorithm == 'RM':
            ready_queue.sort(key=lambda x: (x[3], x[0]))
        elif algorithm == 'DM':
            ready_queue.sort(key=lambda x: (x[4], x[0]))
        elif algorithm == 'EDF':
            ready_queue.sort(key=lambda x: (x[2], x[0]))

        queue_index = 0
        for core in range(num_cores):
            if queue_index < len(ready_queue):
                current_job = ready_queue[queue_index]
                schedule[core].append(current_job[0])
                current_job[1] -= 1
                busy_cycles += 1
                if current_job[1] == 0:
                    completed_jobs += 1
                    ready_queue.pop(queue_index)
                else:
                    queue_index += 1
            else:
                schedule[core].append('IDLE')

    cpu_utilization = (busy_cycles / (num_cores * total_time)) * 100
    return completed_jobs, deadline_misses, cpu_utilization, schedule

```

Figure 1: Implementation and output: Simulator input and priority rules.

```python

def plot_gantt_chart(algorithm, num_cores, schedule, tasks):
    fig, ax = plt.subplots(figsize=(12, 4))
    task_names = [task[0] for task in tasks]
    color_palette = ['#ff7f0e', '#1f77b4', '#2ca02c', '#d62728', '#9467bd', '#8c564b']
    colors = {name: color for name, color in zip(task_names, color_palette)}

    ax.set_title(f"{algorithm} Scheduling - {num_cores} Core(s)")
    ax.set_ylim(0, num_cores * 10 + 10)
    ax.set_xlim(0, len(schedule[0]))
    ax.set_yticks([5 + i * 10 for i in range(num_cores)])
    ax.set_yticklabels([f"Core {i+1}" for i in range(num_cores)])
    ax.grid(True, axis='x', linestyle='--', alpha=0.6)

    for core in range(num_cores):
        start = 0
        while start < len(schedule[core]):
            value = schedule[core][start]
            end = start
            while end < len(schedule[core]) and schedule[core][end] == value:
                end += 1
            if value != 'IDLE':
                ax.broken_barh([(start, end - start)], (1 + core * 10, 8), facecolors=colors[value], edgecolor='black')
                ax.text(start + (end - start) / 2, 5 + core * 10, value, ha='center', va='center', color='white', fontsize=9)
            start = end
    plt.tight_layout()
    plt.savefig(f"{algorithm}_{num_cores}C.png", dpi=150)
    plt.close()

def main():
    with open('input.txt', 'r') as input_file:
        lines = input_file.read().splitlines()
    total_time, num_cores = map(int, lines[0].split())
    num_tasks = int(lines[1])

    tasks = []
    for i in range(num_tasks):
        parts = lines[2 + i].split()
        tasks.append((parts[0], int(parts[1]), int(parts[2]), int(parts[3])))

    output_text = ""
    for algorithm in ['RM', 'DM', 'EDF']:
        completed, misses, utilization, schedule = simulate_scheduling(algorithm, num_cores, total_time, tasks)
        plot_gantt_chart(algorithm, num_cores, schedule, tasks)
        output_text += f"========\nAlgorithm: {algorithm}\nCores: {num_cores}\nCompleted Jobs: {completed}\nDeadline Misses: {misses}\nCPU Utilization: {utilization:.2f}%\n"
        for core in range(num_cores):
            output_text += f"Core {core+1} Schedule:\n" + " ".join(schedule[core]) + "\n"

    with open('output.txt', 'w') as output_file:
        output_file.write(output_text)

    print("The simulation is done and the outputs have been saved")

if __name__ == '__main__':
    main()
```

Figure 2: Implementation and output: Simulator input and priority rules.

9 Recorded one-core and two-core outcomes

Policy	Cores	Completed jobs	Deadline misses	Utilization over recorded horizon
RM	1	17	0	83.33 percent
DM	1	17	0	83.33 percent
EDF	1	17	0	83.33 percent
RM	2	17	0	41.67 percent
DM	2	17	0	41.67 percent
EDF	2	17	0	41.67 percent

The run releases eight T1 jobs, four T2 jobs, and five T3 jobs: 17 jobs requiring 8+12+5=25 core-time units. Dividing by 30 yields 83.33 percent for one core, and dividing by 60 gives 41.67 percent averaged across two cores. This normalized utilization falls because capacity doubles; the supplied work does not halve. Aggregate periodic offered utilization is instead 1/4+3/8+1/7=43/56, approximately 76.79 percent. A finite horizon can differ from that long-run ratio.

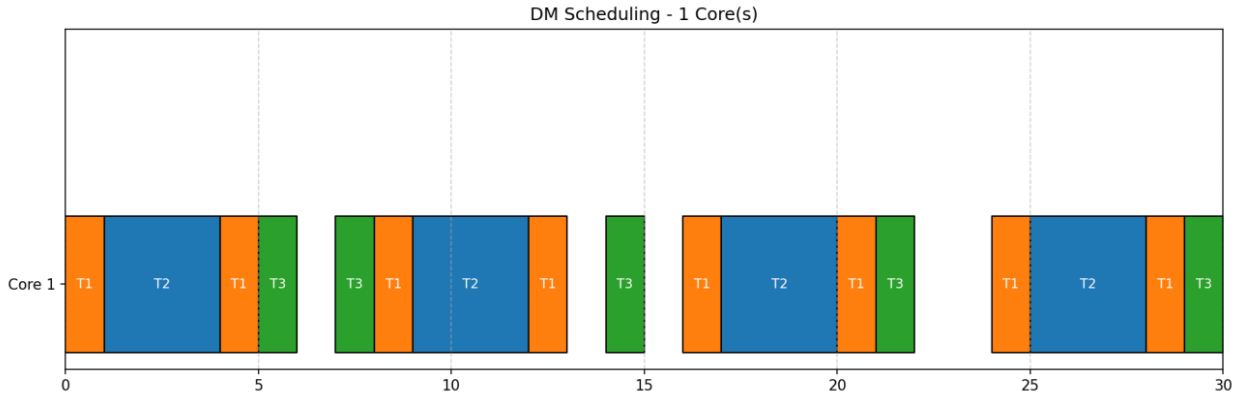


Figure 3: Diagram or recorded result: Recorded one-core and two-core outcomes.

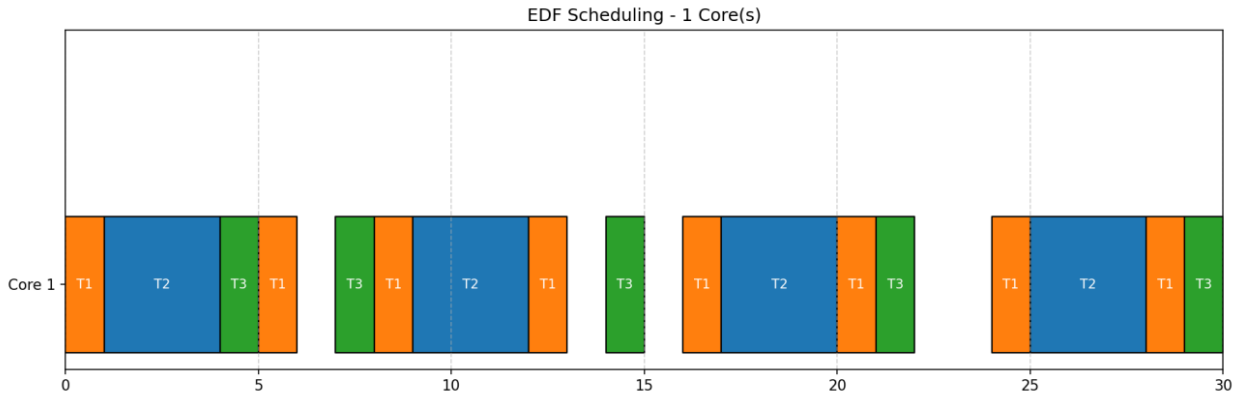


Figure 4: Diagram or recorded result: Recorded one-core and two-core outcomes.

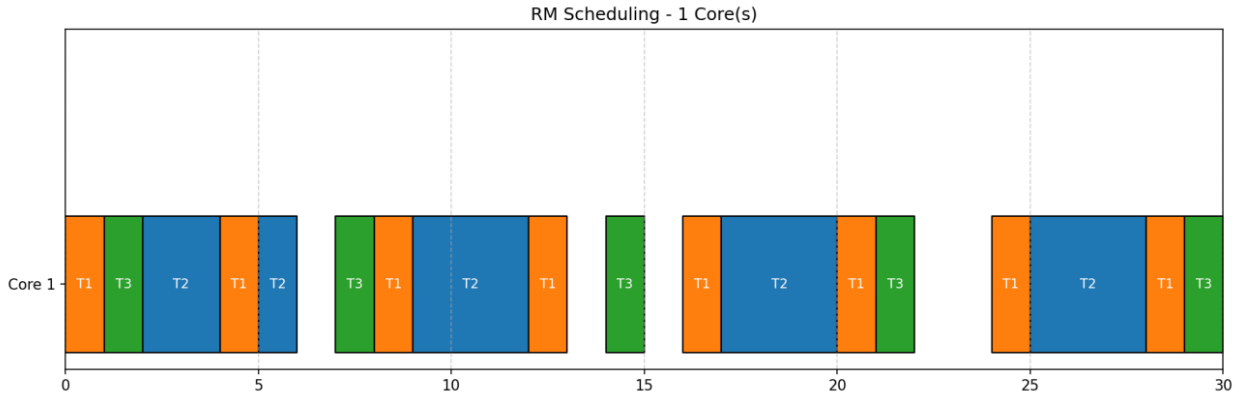


Figure 5: Diagram or recorded result: Recorded one-core and two-core outcomes.

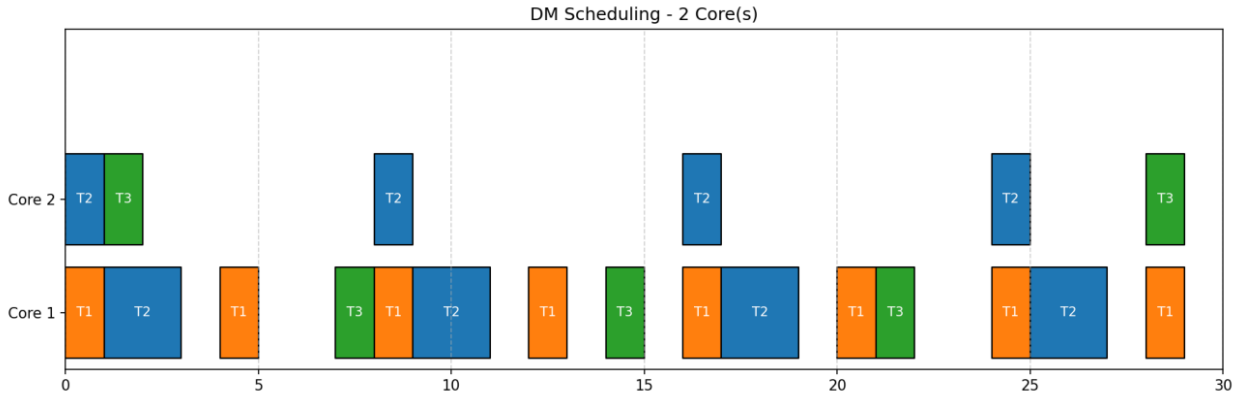


Figure 6: Diagram or recorded result: Recorded one-core and two-core outcomes.

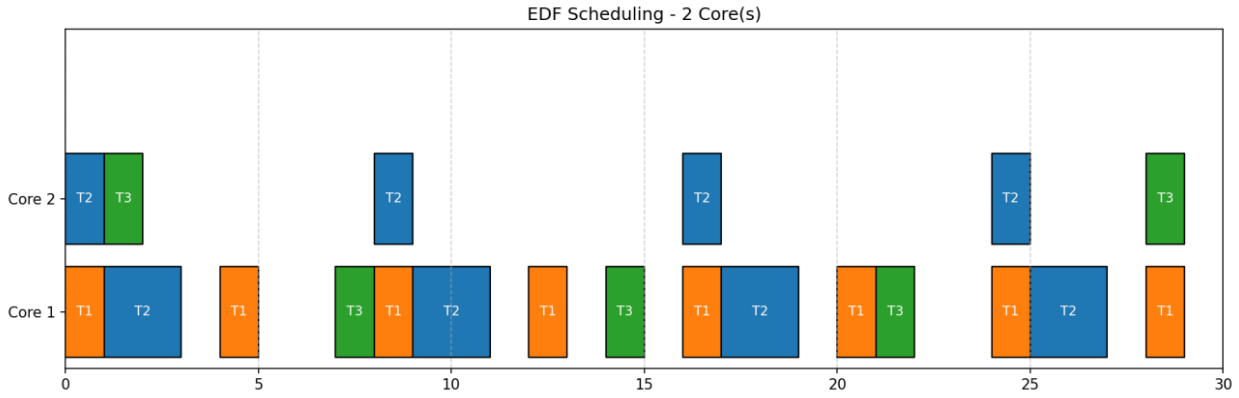


Figure 7: Diagram or recorded result: Recorded one-core and two-core outcomes.

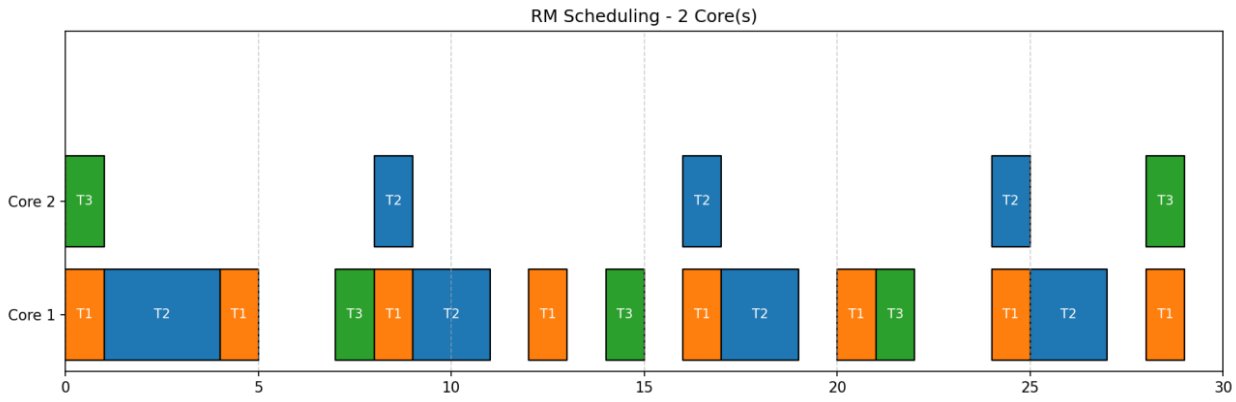


Figure 8: Diagram or recorded result: Recorded one-core and two-core outcomes.

10 Schedulability analysis

RM and DM differ in their choice between T2 and T3, visible at the beginning of the one-core trace. Two cores permit jobs to overlap and add idle capacity, but deadline misses remain zero because the one-core run already completed every released job in the recorded interval. The EDF utilization guarantee applies to independent, preemptible periodic tasks with implicit deadlines under the stated execution assumptions. T2 has D<T, so the familiar implicit-deadline utilization-only test is insufficient here. Fixed-priority analysis requires an appropriate response-time test including interference and blocking assumptions; EDF requires a demand analysis for constrained deadlines. Zero misses in a 30-unit simulation do not prove indefinite schedulability. Similarly, the simulation does not measure power, so it cannot prove that one core is energy-optimal. The evidence supports a smaller hardware-capacity requirement for this observed workload and additional spare capacity with two cores.